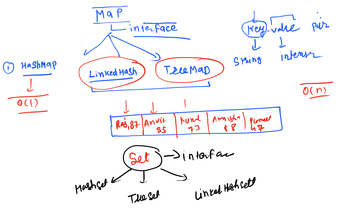
Lec-24

01/07/26 10:31 AM



350. Intersection of Two Arrays II

Solved

Easy | Topics | Companies

Given two integer arrays `nums1` and `nums2`, return *an array of their intersection*. Each element in the result must appear as many times as it shows in both arrays and you may return the result in **any order**.

Example 1:

```
Input: nums1 = [1,2,2,1], nums2 = [2,2]
Output: [2,2]
```

```
for (int i = 0; i < arr2.length; i++) {
        if(map.containsKey(arr2[i]) && map.get(arr2[i])>0) {
            list.add(arr2[i]);
            map.put(arr2[i], map.get(arr2[i]) - 1);
        }
    }
```

```
public static int Longest_Sequence(int[] arr) {
    HashSet<Integer> set = new HashSet<>();
    int ans = 0;
    for (int i = 0; i < arr.length; i++) {
        set.add(arr[i]);
    }
    for (int i = 0; i < arr.length; i++) {
        int x = arr[i];
        if (set.contains(x) && !set.contains(x - 1)) {
            int c = 0;
            while (set.contains(x)) {
                c++;
                set.remove(x);
                x++;
            }
            ans = Math.max(ans, c);
        }
    }
}
```

Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
1->4->5,
1->3->4,
2->6
]

```
ListNode dummy = new ListNode();
        ListNode temp=dummy;
        while(!pq.isEmpty()) {
            ListNode nn=pq.poll();
            dummy.next=nn;
            dummy=dummy.next;
            if(nn.next!=null) {
                pq.add(nn.next);
            }
        }
        return temp.next;
```

```
pq.add(intervals[0]);
        for (int i = 1; i < intervals.length; i++) {
            if(pq.peek()[1]<=intervals[i][0]) {
                pq.poll();
            }
            pq.add(intervals[i]);
        }
```

Example 1:

Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Explanation:

- There is no string in strs that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

```
HashMap<String, List<String>> map = new HashMap<>();
        for (int i = 0; i < arr.length; i++) {
            String s = arr[i];
            String key = GetKey(s);
            if (!map.containsKey(key)) {
                map.put(key, new ArrayList<>());
            }
            map.get(key).add(s);
        }
```